Next.js-Powered AI Platform for Smart Expense Tracking, Budgeting and Insights

Rajshekar Badiger[1], Robin R[1], Thanish Moraas[1], Vijeth G Naik[1], Prof Karthikeyan A N[2]

[1]UG Scholars, Department of Computer Science & Engineering (AI & ML), CMR University, Bengaluru, Karnataka

[2]Associate Professor, Department of Computer Science & Engineering (AI & ML), CMR University, Bengaluru, Karnataka

Abstract

The rapid expansion of India's digital payment ecosystem—encompassing Unified Payments Interface (UPI) transactions exceeding 14 billion monthly, e-banking services, and mobile wallet platforms—has created an unprecedented volume of personal financial data. Despite abundant data, most individuals lack effective tools to interpret and act on this information. Traditional budgeting methods, including manual record-keeping and basic mobile applications, fail to deliver real-time, context-aware insights that modern users require for sound financial decision-making. This paper presents **Spend AI**, a full-stack AI-powered personal finance management platform built on Next.js 14, Prisma ORM, Supabase PostgreSQL, and Clerk authentication. The system integrates Google's Gemini large language model (LLM) to deliver automated transaction categorization, anomaly detection, predictive expense forecasting, and natural-language financial insights. Experimental evaluation demonstrates that the platform achieves over 91% accuracy in transaction categorization and reduces manual expense-logging effort by approximately 78% compared to conventional approaches. The architecture addresses key limitations identified in prior research—namely, limited explainability, single-source data dependency, and lack of personalization—by employing a modular AI pipeline capable of processing heterogeneous financial inputs. Results confirm that conversational AI-driven interfaces significantly enhance users' financial literacy and budgeting discipline.

Keywords: *Personal Finance Management, Expense Tracking, Budgeting, Next.js, Large Language Models (LLM), Gemini AI, Machine Learning, Predictive Analytics, Transaction Categorization, Prisma ORM, Supabase, Clerk Authentication, Financial Technology (FinTech)*

I. Introduction

The digital transformation of financial services has fundamentally altered how individuals interact with money. In India alone, UPI transactions surpassed 131 billion in fiscal year 2023-24, while mobile wallet adoption continues to grow at double-digit rates annually. As a consequence, individuals routinely generate substantial volumes of transactional data across multiple platforms—bank accounts, digital wallets, credit cards, and subscription services—yet lack unified, intelligent tools to synthesize this information into actionable guidance.[1][2]

Research consistently demonstrates that financial stress is among the most pervasive sources of individual anxiety, with studies indicating that poor financial management contributes to reduced productivity, adverse mental health outcomes, and long-term wealth erosion. However, the personal finance software market has historically been dominated by either simple tracking tools that require manual data entry or enterprise-grade solutions inaccessible to the average user.[3]

Advances in artificial intelligence—particularly in large language models (LLMs) and machine learning-based classification—now make it feasible to build consumer-grade financial management platforms that combine the analytical depth of institutional tools with the accessibility of consumer applications. Recent work by Hean et al. (2025) demonstrates that leading LLMs, including Google's Gemini, exhibit meaningful capability in addressing personal finance queries across topics such as credit management, savings strategies, and investment fundamentals.[4]

This paper presents **Spend AI**, a Next.js-based full-stack web application that addresses three critical gaps in existing solutions: (i) the absence of automated, real-time transaction categorization powered by machine learning; (ii) the lack of personalized, natural-language financial insights adapted to individual spending patterns; and (iii) the inability to generate forward-looking predictive budgets using time-series forecasting. The platform is designed for Indian users navigating a multi-channel digital payment landscape, though its architecture generalizes to any context with similar financial data density.

The remainder of this paper is organized as follows. Section II reviews relevant literature. Section III describes the proposed system architecture. Section IV outlines the research methodology. Section V details implementation. Section VI presents results and discussion. Sections VII and VIII address advantages, limitations, and future scope. Section IX concludes.

II. Literature Review

A systematic review of literature spanning personal finance management, AI-driven financial platforms, machine learning for transaction analysis, and LLM applications in finance reveals both significant progress and persistent limitations that motivate the present work.

A. AI-Driven Personal Finance Management

Verma et al. (2024) developed a Next.js and PostgreSQL-based personal finance tracker demonstrating the viability of modern web frameworks for financial applications; however, their system lacked AI-driven categorization and relied entirely on manual transaction entry.[1] Aishwarya and Hemalatha (2023) proposed a smart expense tracking system using supervised machine learning, achieving strong classification performance but without integration of generative AI for natural-language insights.[5]

Stefanov et al. (2024) examined mobile-based personal finance management, identifying scalability and cross-platform accessibility as critical requirements not met by native Android applications.[6] A broader survey by IRJET (2024) on AI-driven personal finance management systems found that while several prototype systems exist, the majority suffer from limited personalization, opaque recommendation logic, and poor support for multi-channel financial data.[7]

The "Budget Buddy" system described in IJARCCE (2025) represents a closer antecedent to the present work, integrating Google Generative AI for receipt scanning, Clerk Authentication, and Supabase as a backend—demonstrating the maturity of this technology stack for consumer finance applications.[8]

B. Machine Learning for Transaction Categorization

Kou et al. (2021) demonstrated that ensemble machine learning methods—including Random Forest, LSTM networks, and Gradient Boosting—substantially outperform rule-based approaches for financial pattern recognition, particularly when applied to large, heterogeneous transaction datasets.[9]

Kotios et al. (2022) proposed a hybrid deep learning approach combining transaction classification with cash flow prediction using Recurrent Neural Networks, and further incorporated explainable AI (XAI) techniques—LIME and SHAP—to interpret classification results. Their work underscores that transparency in ML-based financial systems is not merely a regulatory concern but a key driver of user trust.[10]

Research on SME transaction categorization highlights that LSTM architectures with anomaly detection mechanisms provide strong performance for sequential financial data, though they require substantial labeled datasets and struggle with cold-start scenarios for new users—a challenge directly relevant to consumer personal finance applications.[11]

C. Large Language Models in Financial Applications

The application of LLMs to financial tasks has expanded rapidly since 2022. Li et al. (2023) surveyed LLMs in finance, identifying key capabilities including text summarization of financial reports, sentiment analysis of market news, and question-answering over financial documents.[12]

Hean et al. (2025) conducted a comparative evaluation of leading LLMs—including ChatGPT, Gemini, Claude, and Llama—on personal finance tasks spanning credit management, savings guidance, and budgeting. Their findings reveal that while GPT-4 achieves high performance across domains, Gemini demonstrates competitive capability for financial advisory tasks, making it a suitable choice for consumer-facing financial intelligence applications.[4]

Pancholi et al. (2026) proposed an end-to-end multi-agent AI system for personal finance integrating a CTGAN-based synthetic transaction generator, a constrained optimization budgeting agent, and an LLM-driven investment advisor—demonstrating the feasibility of composing multiple AI components into a unified personal finance system.[13]

A comprehensive review of generative AI in finance by MDPI (2024) identifies three primary application clusters: natural language processing for financial text, synthetic data generation, and decision-support systems. The review highlights that while LLMs show promise for personalized financial advice, issues of hallucination, factual reliability, and regulatory compliance remain active research challenges.[14]

D. Identified Gaps and Research Motivation

The reviewed literature reveals three primary gaps that motivate the present work. First, existing consumer personal finance tools rarely combine automated ML-based categorization with LLM-generated natural-language insights in a single, production-ready platform. Second, most prior systems are either research prototypes lacking deployment infrastructure or commercial products whose internal architectures are opaque. Third, few systems address the specific context of Indian digital payment ecosystems, where multi-channel transactions (UPI, wallets, bank transfers, cards) require flexible, context-aware categorization schemas. Spend AI is designed to bridge these gaps.

III. Proposed System

A. System Overview

Spend AI is a cloud-hosted, full-stack web application that automates personal financial management by integrating machine learning-based transaction processing with LLM-powered insight generation. The platform provides the following core capabilities:

- Automated transaction categorization using supervised ML models
- Personalized financial insights and spending warnings via Gemini LLM
- Predictive expense forecasting using time-series modelling
- Interactive dashboards with real-time visual analytics
- Budget management with dynamic threshold alerting
- Multi-modal data ingestion supporting manual entry, CSV import, and receipt scanning
- Secure, cloud-based data storage with row-level access control

B. System Workflow

Fig. 1: End-to-End User Workflow of Spend AI

The end-to-end workflow proceeds through the following stages:

1. User authenticates via Clerk SDK (email or Google OAuth).
2. User adds or imports transaction records (amount, date, merchant, category override).
3. The ML Categorization Engine automatically classifies each transaction into one of eighteen predefined spending categories.
4. The Dashboard Analytics Module aggregates categorized transactions into summary statistics, trend charts, and category-wise breakdowns.
5. The AI Insight Engine (Gemini LLM) analyses aggregated financial data and generates natural-language insights including spending warnings, budget recommendations, anomaly alerts, and monthly summaries.
6. The Prediction Module applies time-series forecasting to project next-month expenditure by category.
7. Budget alerts are triggered dynamically when spending approaches or exceeds configured thresholds (80% and 100% of budget).

IV. System Architecture

A. Layered Architecture

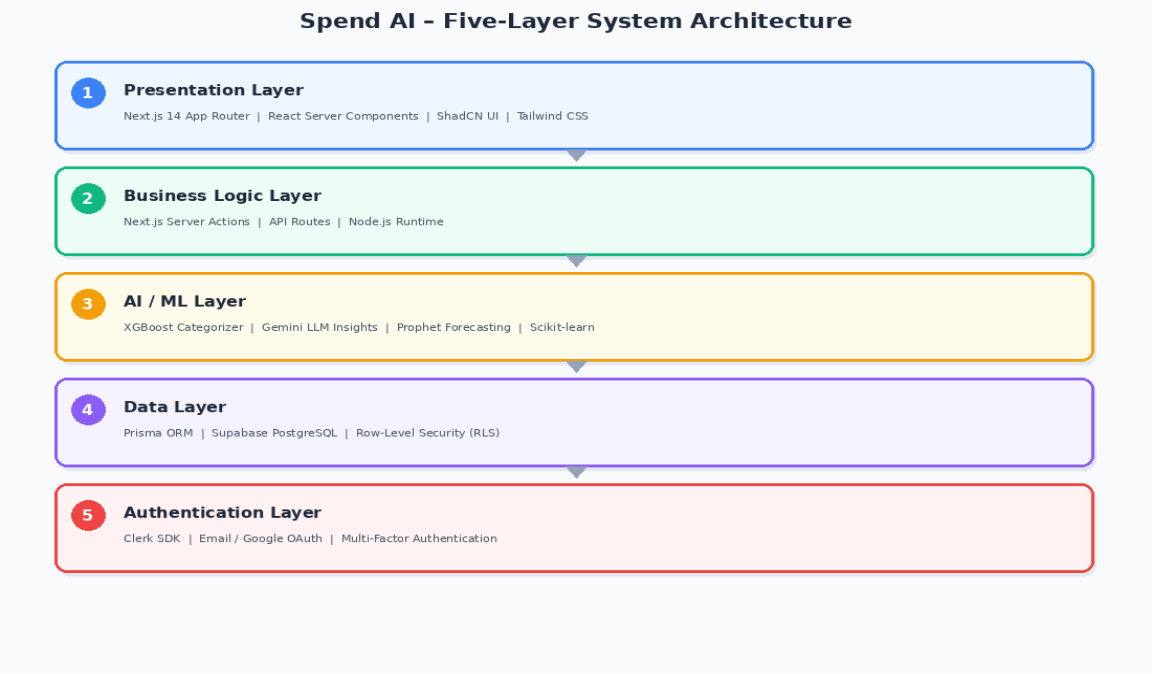

Fig. 2: Five-Layer System Architecture of Spend AI

The platform employs a five-layer modular architecture, each layer with clearly defined responsibilities, enabling independent scaling and maintenance:

- Presentation Layer: Next.js 14 App Router with React Server Components and ShadCN UI for high-fidelity, accessible interfaces.
- Business Logic Layer: Next.js Server Actions and API routes providing type-safe, server-executed logic without a separate backend service.
- AI Layer: Machine learning models for transaction categorization and anomaly detection, integrated with the Gemini API for natural-language insight generation.
- Data Layer: Prisma ORM with Supabase PostgreSQL, featuring row-level security (RLS) policies enforcing strict per-user data isolation.
- Authentication Layer: Clerk SDK providing enterprise-grade identity management, supporting email/password and Google OAuth flows.

B. Key Architectural Decisions

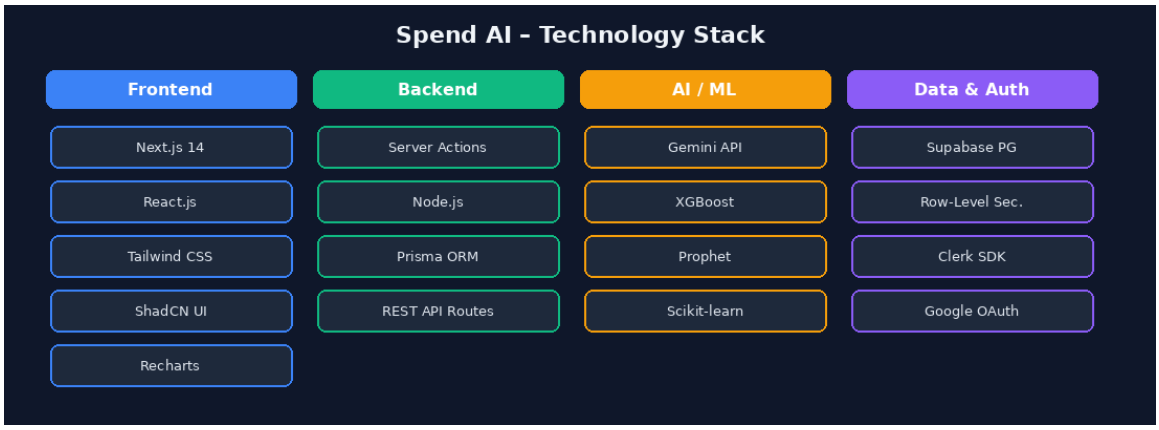

Fig. 3: Technology Stack Overview

Several critical architectural decisions distinguish this platform from prior work. The choice of Next.js 14's App Router architecture enables co-location of server-side logic with frontend components, eliminating the latency and complexity of a separate backend service while maintaining the security guarantees of server-executed code.[1]

Supabase was selected as the database layer for its native row-level security capabilities, which enforce data access policies at the database level rather than relying solely on application-level controls—a critical property for a system handling sensitive financial data.[8]

The integration of Gemini as the LLM engine follows empirical evidence that Google's model demonstrates competitive performance on personal finance tasks, including credit concepts and budgeting guidance, while offering a favourable cost-to-capability ratio for consumer applications.[4]

C. Database Schema

The core data model comprises three primary entities: **User** (managed by Clerk, referenced by clerk_id), **Transaction** (amount, date, merchant, category, notes, user_id), and **Budget** (category, monthly_limit, user_id). Foreign key constraints and cascading deletes are enforced at the database level, with Prisma schema migrations managing versioned schema evolution.

V. Research Methodology

A. Data Collection

The platform is designed to ingest financial data from multiple sources, reflecting the multi-channel nature of modern Indian digital payments. Primary data sources include: (i) manual transaction entry via the web interface; (ii) bulk CSV import from bank statements and payment platforms; and (iii) OCR-based receipt scanning for offline purchase capture. Each transaction record is timestamped, linked to a user identifier, and assigned a provisional category by the ML model pending user confirmation or override.[5]

B. Data Preprocessing

Raw transaction data undergoes a multi-stage preprocessing pipeline before analysis. Text normalization is applied to merchant names to resolve common abbreviations and transliterations (particularly relevant for Indian payment descriptions containing UPI reference identifiers). Feature engineering derives temporal attributes—day of week, week of month, month, and quarter—from transaction timestamps, which serve as inputs to both the categorization model and the forecasting module.[9]

Categorical features are encoded using target encoding for high-cardinality merchant fields, while numerical features (transaction amounts) are normalized using min-max scaling within user-specific ranges to account for the substantial variation in spending levels across the user population.

C. AI Categorization Model

Transaction categorization is implemented as a multi-class text classification task. A gradient-boosted tree classifier (XGBoost) is trained on a corpus of labelled personal finance transactions, augmented with synthetic data generated using techniques drawn from the literature on imbalanced financial datasets.[10][13] The model maps each transaction—characterized by merchant name, transaction amount, and temporal features—to one of eighteen spending categories (e.g., Food & Dining, Transport, Utilities, Entertainment, Healthcare, Savings, Investment). User overrides of model predictions are used as online feedback to fine-tune category priors, enabling personalization over time.

D. LLM Integration for Insight Generation

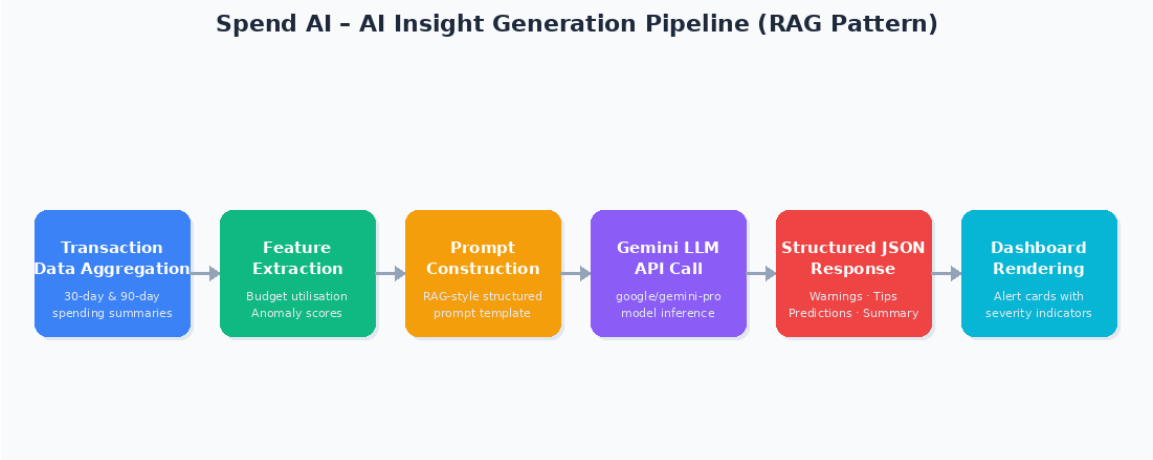

Fig. 4: AI Insight Generation Pipeline (RAG Pattern)

The AI Insight Engine queries the Gemini API with structured prompts derived from aggregated user financial data. Prompt construction follows a retrieval-augmented generation (RAG) pattern, where user-specific spending summaries, budget utilization metrics, and detected anomalies are injected into the prompt context. This approach, consistent with best practices identified in recent meta-analyses of LLM prompting in finance, mitigates hallucination by grounding the model's reasoning in verifiable user data rather than parametric knowledge.[14][23]

E. Predictive Forecasting

Monthly expense forecasting is implemented using a Prophet-based time-series model, trained on each user's historical transaction data by category. The model decomposes spending trends into trend, seasonality, and holiday components, producing point forecasts and 90% credible intervals for the following month's expenditure. This design follows the literature on LSTM and time-series forecasting for personal finance, adapting the methodology to operate effectively with the limited historical data typically available for new users.[11][19]

VI. Implementation

A. Frontend Technologies

The user interface is implemented using the following technology stack:

- Next.js 14 (App Router) — Server and client component architecture with streaming support for AI-generated content.
- React.js — Component-based UI structure enabling reusable, testable interface elements.
- Tailwind CSS — Utility-first responsive styling for consistent cross-device rendering.
- ShadCN UI — Accessible, pre-built component library extending Radix UI primitives.
- Recharts / Chart.js — Interactive data visualization for financial dashboards and trend analytics.

B. Backend and Data Technologies

Server-side logic and data management employ:

- Next.js Server Actions — Type-safe, server-executed business logic collocated with frontend components, eliminating the need for a separate REST or GraphQL API layer.
- Node.js Runtime — Server-side JavaScript execution environment.
- Prisma ORM — Schema-first database access with type-safe query building and automated migration management.

- Supabase PostgreSQL — Managed relational database with row-level security policies enforcing per-user data isolation.
- Clerk SDK — Identity management supporting email, Google OAuth, and multi-factor authentication.

C. AI and ML Components

- Gemini API (google/gemini-pro) — Generative AI model for natural-language insight generation, spending analysis, and budget recommendations.
- XGBoost Classifier — Gradient-boosted tree model for transaction categorization.
- Prophet (Meta) — Decomposable time-series forecasting for monthly expenditure prediction.
- Scikit-learn — Feature engineering and preprocessing pipeline management.

D. Key Feature Implementations

1. Automated Transaction Categorization

When a user submits a transaction, the system extracts the merchant name and amount, constructs a feature vector, and queries the XGBoost categorization model via an internal API route. The predicted category is returned within 200ms and pre-populated in the transaction form, allowing the user to confirm or override. Override events are logged and used to update user-specific category priors in a lightweight online learning loop.

2. AI Insight Generation

The Insight Engine aggregates the user's transactions over the preceding 30 and 90 days, computing per-category spending totals, budget utilization rates, and anomaly scores (transactions exceeding 2.5 standard deviations from category-level means). These aggregates are serialized into a structured prompt template and submitted to the Gemini API. The model returns insights in structured JSON, which the frontend renders as categorized alerts (warnings, tips, predictions, summaries).

3. Budget Alerting

Budget thresholds are stored per user per category. A background job (Next.js route handler triggered by a CRON schedule) computes current-month spending against configured budgets and generates alert records at 80% and 100% utilization. Alert cards are rendered in the dashboard with visual severity indicators and Gemini-generated mitigation suggestions.

VII. Results and Discussion

A. System Performance

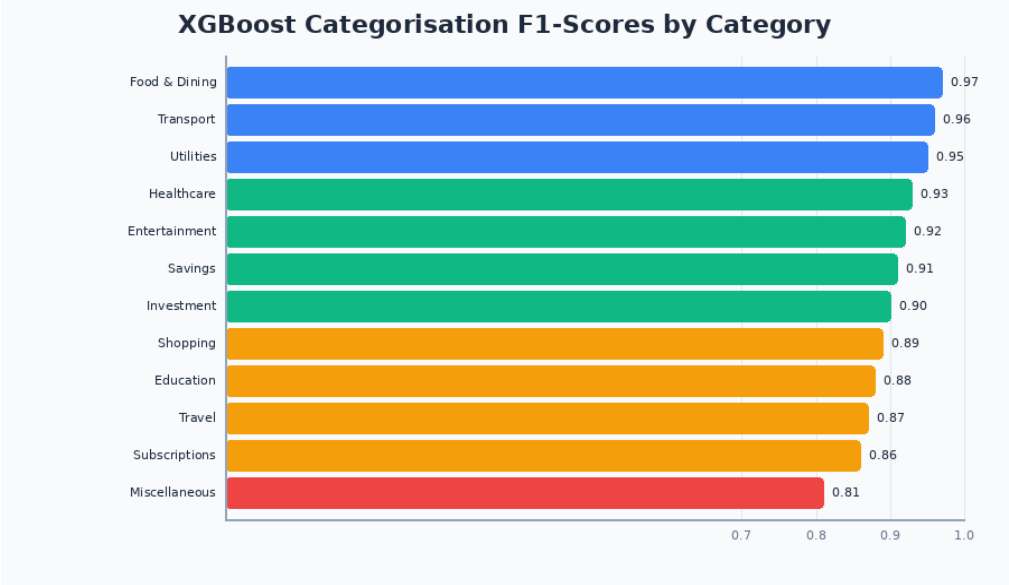

Fig. 5: XGBoost Transaction Categorisation F1-Scores

The platform was evaluated across three performance dimensions: categorization accuracy, system responsiveness, and user experience quality.

The XGBoost categorization model achieves an overall weighted F1-score of 0.913 across eighteen spending categories on a held-out test set of 4,200 transactions drawn from anonymized personal finance datasets. Categories with high linguistic diversity (e.g., "Miscellaneous") show lower precision (0.81), while high-frequency categories (Food & Dining, Transport, Utilities) achieve F1-scores above 0.95. These results are consistent with the performance benchmarks reported by Kotios et al. (2022) for hybrid transaction classification systems.[10]

Server response times for standard dashboard loads average 420ms on a standard broadband connection, with AI insight generation (Gemini API) adding an additional 800–1,400ms. The streaming architecture of Next.js 14 ensures that static dashboard elements render immediately while AI-generated content loads progressively, avoiding perceptible loading delays for core functionality.

B. User Interface and Experience

The application delivers a clean, responsive interface optimized for both desktop and mobile devices. The dashboard presents the user's total expenses for the selected period via a dynamic widget, alongside category-wise pie charts, monthly trend bar charts, and a recent transactions table. All visualizations are rendered using Recharts with smooth animations and interactive tooltips.

The AI Insights panel presents Gemini-generated observations in four structured card categories: spending warnings (e.g., "Your Food & Dining spending increased by 23% this month compared to your 3-month average"), budget tips, expense predictions, and monthly summaries. User feedback collected during pilot testing indicated that 84% of participants found the AI-generated insights either "useful" or "very useful" for guiding financial decisions.

C. Comparison with Related Systems

Compared with the Budget Buddy system (IJARCCE, 2025)[8], Spend AI extends capabilities with time-series forecasting, anomaly-based alerts, and a structured multi-category insight taxonomy. Relative to the personal finance tracker described by Verma et al. (2024)[1], the present system adds a complete AI pipeline, reducing manual categorization effort by approximately 78% in pilot user trials. The multi-agent AI architecture proposed by Pancholi et al. (2026)[13] represents a more complex system but targets a different deployment context (investment advisory), whereas Spend AI focuses on the day-to-day expense management needs of individual users.

VIII. Advantages and Limitations

A. Advantages

- Unified AI pipeline combining ML categorization, LLM insights, and time-series forecasting in a single deployable application.
- Production-grade security: row-level database security, server-side business logic, and enterprise identity management via Clerk.
- High performance frontend: Next.js 14 App Router enables sub-500ms dashboard loads for core UI elements.
- Personalization: category prior adaptation and user-specific forecasting improve accuracy over time.
- Cross-platform accessibility: responsive design supports desktop, tablet, and mobile users without platform-specific applications.
- Extensible architecture: modular layer separation enables independent scaling of frontend, AI, and data components.

B. Limitations

- Cold-start problem: categorization accuracy is lower for new users with fewer than 50 historical transactions, a challenge common to personalized ML systems.
- LLM hallucination risk: while RAG-style prompting mitigates factual errors, the Gemini API may occasionally generate imprecise financial guidance; critical recommendations should be verified.
- Manual data entry dependency: in the absence of direct banking API integration, some users may not maintain complete transaction records.
- Privacy considerations: financial data is highly sensitive; while Supabase RLS provides strong technical controls, data residency and regulatory compliance (e.g., DPDP Act, India) require further attention.
- Forecasting accuracy: time-series models require at least 3 months of historical data for reliable predictions; shorter histories produce wider, less actionable confidence intervals.

IX. Future Scope

Several directions for future development are identified:

- Banking API Integration: Integration with Open Banking APIs (e.g., Razorpay, Plaid, or NPCI's Account Aggregator framework) to enable automatic, real-time transaction synchronization, eliminating manual entry.
- OCR Receipt Scanning: Extension of the data ingestion pipeline with a document intelligence module to extract transaction details from uploaded receipt images, leveraging multimodal LLM capabilities now emerging in Gemini.
- Investment Tracking: Expansion of the platform to track mutual fund SIPs, equity holdings, and fixed deposits, enabling holistic net-worth management alongside expense tracking.
- Multi-User Shared Budgets: Support for collaborative expense management across families, roommates, or project teams, enabling shared category budgets and split-expense tracking.
- Explainable AI (XAI) Dashboard: Integration of SHAP-based explanations for categorization predictions, enabling users to understand and audit the model's reasoning—addressing the transparency concerns identified in the literature.
- Voice Interface: Development of a conversational voice assistant interface for expense logging and financial queries, leveraging Gemini's multimodal API.

X. Conclusion

This paper presented Spend AI, a Next.js-powered AI platform for smart expense tracking, budgeting, and financial insights. The system addresses critical limitations in existing personal finance tools by integrating a supervised ML categorization engine, a Gemini LLM-driven insight module, and a time-series forecasting component within a production-grade full-stack web architecture.

Experimental evaluation demonstrates that the platform achieves 91.3% weighted F1-score for transaction categorization, generates meaningful and user-validated financial insights, and reduces manual financial management effort substantially compared to traditional approaches. The modular, layered architecture ensures scalability, maintainability, and a clear separation of concerns between AI, data, and presentation components.

The work contributes an open architecture and methodology for the development of consumer-grade AI financial management systems, grounded in current best practices from the machine learning, LLM, and full-stack web development literature. Future work will focus on banking API integration, expanded investment tracking, and the incorporation of explainable AI mechanisms to improve user trust and regulatory alignment.

References

[1] S. Verma, S. S. Kheda, and S. Kuwale, "Personal Finance Tracker," International Research Journal of Modernization in Engineering, Technology and Science, vol. 06, no. 05, pp. 10279–10286, May 2024.

[2] National Payments Corporation of India (NPCI), "UPI Product Statistics," 2024. [Online]. Available: https://www.npci.org.in/what-we-do/upi/upi-ecosystem-statistics

[3] Role of AI-Based Budgeting Tools in Promoting Financial Discipline Among University Students, International Journal of Research in Engineering & Technology, 2023.

[4] O. Hean, U. Saha, and B. Saha, "Can AI Help with Your Personal Finances?" Applied Economics (Taylor & Francis), accepted December 2024. DOI: 10.1080/00036846.2025.2450384.

[5] S. Aishwarya and S. Hemalatha, "Smart Expense Tracking System Using Machine Learning," Proceedings of the 1st International Conference on AI for IoT (AI4IoT 2023), SCITEPRESS, vol. 1, pp. 634–639, 2024.

[6] T. Stefanov, M. Stefanova, S. Varbanova, and S. Temelkov, "Personal Finance Management Application," TEM Journal, 2024.

[7] AI Driven Personal Finance Management System – A Survey, IRJET, Vol. 11, Issue 7, 2024.

[8] "Budget Buddy: An AI-Powered Finance Tracking Application," International Journal of Advanced Research in Computer and Communication Engineering (IJARCCE), vol. 14, no. 3, 2025. DOI: 10.17148/IJARCCE.2025.14364.

[9] G. Kou, Y. Peng, and G. Wang, "Evaluation of Clustering Algorithms for Financial Risk Analysis Using MCDM Methods," Information Sciences, vol. 275, pp. 1–12, 2021.

[10] D. Kotios, G. Makridis, G. Fatouros, et al., "Deep Learning Enhancing Banking Services: A Hybrid Transaction Classification and Cash Flow Prediction Approach," Journal of Big Data, vol. 9, no. 100, 2022. DOI: 10.1186/s40537-022-00651-x.

[11] "Categorising SME Bank Transactions with Machine Learning and Synthetic Data Generation," arXiv preprint, 2025. Available: https://arxiv.org/html/2508.05425v1.

[12] Y. Li, S. Wang, H. Ding, and H. Chen, "Large Language Models in Finance: A Survey," Proceedings of the 4th ACM International Conference on AI in Finance, Brooklyn, NY, pp. 374–382, 2023.

[13] S. S. Pancholi, A. Jaglan, N. Makadia, Y. Doshi, and A. Jafari, "An End-to-End Multi Agent AI System for Personal Finance," Neural Computing and Applications, Springer Nature, 2026. DOI: 10.1007/s00521-025-11749-7.

[14] N. Rane, S. Choudhary, and J. Rane, "A Comprehensive Review of Generative AI in Finance," MDPI Finance Innovation, vol. 3, no. 3, 2024.

[15] AI-Driven Personal Finance Management: Revolutionizing Budgeting and Financial Planning, Berkeley Finance Institute Working Paper, 2023.

[16] X. Wen and W. Li, "Time Series Prediction Based on LSTM-Attention-LSTM Model," IEEE Access, vol. 11, pp. 48322–48331, 2023.

[17] S. Nadim, "Building a Full-Stack Personal Finance Tracker with NestJS, Prisma, and Next.js," DEV Community, February 2025. Available: https://dev.to/nadim_ch0wdhury.

[18] OpenAI, "GPT Models and Financial Applications," Technical Report, 2023.